Restaurant Case Study UseCase (Using Hadoop and PySpark)

```
File  Edit  View  Search  Terminal  Help
tle Tokyo, Legaspi Village, Makati City, Makati City",121.014101,14.553708,Japan
ese,1200,Botswana Pula(P),Yes,No,No,No,3,4.5,Dark Green,Excellent,591
6300002,Heat - Edsa Shangri-La,162,Mandaluyong City,"Edsa Shangri-La, 1 Garden W
ay, Ortigas, Mandaluyong City","Edsa Shangri-La, Ortigas, Mandaluyong City","Eds
a Shangri-La, Ortigas, Mandaluyong City, Mandaluyong City",121.056831,14.581404,
"Seafood, Asian, Filipino, Indian",4000,Botswana Pula(P),Yes,No,No,No,4,4.4,Gree
n,Very Good,270
6318506,Ooma,162,Mandaluyong City,"Third Floor, Mega Fashion Hall, SM Megamall,
Ortigas, Mandaluyong City","SM Megamall, Ortigas, Mandaluyong City","SM Megamall
, Ortigas, Mandaluyong City, Mandaluyong City",121.056475,14.585318,"Japanese, S
ushi",1500,Botswana Pula(P),No,No,No,No,4,4.9,Dark Green,Excellent,365
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/zomato
[cloudera@quickstart ~]$ hdfs dfs -put /home/cloudera/Desktop/zomato.csv /user/c
loudera/zomato/
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/zomato/
Found 1 items
-rw-r--r--   1 cloudera cloudera    2249719 2026-03-20 22:14 /user/cloudera/zoma
to/zomato.csv
[cloudera@quickstart ~]$
```

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
Using Python version 2.6.6 (r266:84292, Jul 23 2015 15:22:56)
SparkSession available as 'spark'.
>>> from pyspark.sql import SparkSession
>>> from pyspark.sql import functions as F
>>> from pyspark.sql.types import *
>>>
>>> spark = SparkSession.builder.appName("ZomatoAnalysis").getOrCreate()
>>> sc = spark.sparkContext
>>>
>>> df = spark.read.option("header", "true").option("inferSchema", "true").option("multiLine", "true").option("escape", '"').csv("h
dfs:///user/cloudera/zomato/zomato.csv")
2026-03-20 22:17:42 WARN  DomainSocketFactory:117 - The short-circuit local reads feature cannot be used because libhadoop cannot b
e loaded.
>>>
>>> df.printSchema()
root
 |-- Restaurant ID: integer (nullable = true)
 |-- Restaurant Name: string (nullable = true)
 |-- Country Code: integer (nullable = true)
```

1. Global Cuisine Popularity Analysis

- The Problem: The Cuisines column contains comma-separated values (e.g., "French, Japanese, Desserts").
- Goal: Calculate the total count of each cuisine globally and per city to identify the most popular food types.

```
File  Edit  View  Search  Terminal  Help
>>>
>>> from pyspark.sql.functions import col, split, explode, trim, lower
>>>
>>> cuisine_df = df.withColumn("Cuisine", explode(split(col("Cuisines"), ","))) \
...                .withColumn("Cuisine", trim(lower(col("Cuisine"))))
>>>
>>> cuisine_df.groupBy("Cuisine").count().orderBy(col("count").desc()).show(20)
[Stage 8:==========================================>            (153 + 4) / 200]
+------------+-----+
|     Cuisine|count|
+------------+-----+
|north indian| 3960|
|     chinese| 2735|
|   fast food| 1986|
|     mughlai|  995|
|     italian|  764|
|      bakery|  745|
| continental|  736|
|        cafe|  703|
[Mozilla Firefox]   cloudera@quickstart:~
```

```
File  Edit  View  Search  Terminal  Help
>>>
>>> cuisine_df.groupBy("City", "Cuisine").count().orderBy("City", col("count").desc()).show(30)
+---------+--------------+-----+
|     City|       Cuisine|count|
+---------+--------------+-----+
|Abu Dhabi|        indian|    7|
|Abu Dhabi|      american|    4|
|Abu Dhabi|  north indian|    3|
|Abu Dhabi|       chinese|    3|
|Abu Dhabi|      japanese|    2|
|Abu Dhabi|         asian|    2|
|Abu Dhabi|         pizza|    2|
|Abu Dhabi|       arabian|    2|
|Abu Dhabi|       italian|    2|
|Abu Dhabi|middle eastern|    2|
|Abu Dhabi|       mexican|    1|
|Abu Dhabi|      filipino|    1|
|Abu Dhabi|       biryani|    1|
|Abu Dhabi|       seafood|    1|
```

2. Normalizing and Comparing Costs Across Currencies

- **The Problem:** The Average Cost for two is in different local currencies (INR, Dollar, Botswana Pula, etc.).
- **Goal:** Determine which city/country has the most expensive or affordable dining scenes on a global scale.

```
File  Edit  View  Search  Terminal  Help
>>>
>>>
>>> from pyspark.sql.functions import col, round as spark_round
>>> from pyspark.sql.types import StructType, StructField, StringType, DoubleType
>>>
>>> exchange_rates = {
...     "Botswana Pula(P)": 0.074,
...     "Indian Rupees(Rs.)": 0.012,
...     "Dollar($)": 1.0,
...     "Emirati Diram(AED)": 0.27,
...     "South African Rand(R)": 0.054,
...     "Brazilian Real(R$)": 0.20,
...     "Turkish Lira(TL)": 0.031,
...     "New Zealand Dollar($)": 0.61,
...     "Pounds(£)": 1.27,
...     "Qatari Rial(QR)": 0.27,
...     "Sri Lankan Rupee(LKR)": 0.0031,
...     "Rand(R)": 0.054,
...     "Indonesian Rupiah(IDR)": 0.000065,
```

Activate Windows

```
File  Edit  View  Search  Terminal  Help
... }
>>>
>>> rates_data = [(k, v) for k, v in exchange_rates.items()]
>>> rates_schema = StructType([
...     StructField("Currency", StringType(), True),
...     StructField("rate_to_usd", DoubleType(), True)
... ])
>>> rates_df = spark.createDataFrame(rates_data, schema=rates_schema)
>>>
>>> from pyspark.sql.functions import broadcast
>>>
>>> cost_df = df.join(broadcast(rates_df), on="Currency", how="left") \
...             .withColumn("Cost_USD", spark_round(col("Average Cost for two").cast("double") * col("rate_to_usd"), 2))
>>>
>>> cost_df.groupBy("City", "Currency") \
...        .agg(F.round(F.avg("Cost_USD"), 2).alias("Avg_Cost_USD")) \
...        .orderBy(col("Avg_Cost_USD").desc()).show(20)
+---------------+-----------------+------------+
|           City|         Currency|Avg_Cost_USD|
```

Activate Windows

```
File  Edit  View  Search  Terminal  Help
...             .withColumn("Cost_USD", spark_round(col("Average Cost for two").cast("double") * col("rate_to_usd"), 2))
>>>
>>> cost_df.groupBy("City", "Currency") \
...         .agg(F.round(F.avg("Cost_USD"), 2).alias("Avg_Cost_USD")) \
...         .orderBy(col("Avg_Cost_USD").desc()).show(20)
+----------------+------------------+------------+
|            City|          Currency|Avg_Cost_USD|
+----------------+------------------+------------+
|      Pasay City|  Botswana Pula(P)|      246.67|
|       Singapore|         Dollar($)|      155.75|
|Mandaluyong City|  Botswana Pula(P)|       148.0|
|     Taguig City|  Botswana Pula(P)|      127.65|
|     Paynesville|         Dollar($)|       120.0|
|   Tagaytay City|  Botswana Pula(P)|        88.8|
|     Makati City|  Botswana Pula(P)|        85.1|
|       Princeton|         Dollar($)|        70.0|
|Vineland Station|         Dollar($)|        70.0|
|      Pasig City|  Botswana Pula(P)|       69.07|
|   San Juan City|  Botswana Pula(P)|        62.9|
```

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
>>> cost_df.groupBy("City").agg(F.round(F.avg("Cost_USD"), 2).alias("Avg_Cost_USD")) \
...         .orderBy(col("Avg_Cost_USD").asc()).show(10)
+---------------+------------+
|           City|Avg_Cost_USD|
+---------------+------------+
|     Birmingham|        null|
|       Auckland|        null|
|Wellington City|        null|
|     Manchester|        null|
|      Edinburgh|        null|
|         London|        null|
|         Miller|         0.0|
|      ??stanbul|        2.53|
|         Ankara|        2.71|
|      Faridabad|        5.37|
+---------------+------------+
only showing top 10 rows

>>> 
```

3. Impact of Services on Customer Ratings

- **The Problem:** Does having "Online Delivery" or "Table Booking" actually lead to better reviews?
- **Goal:** Provide business insights into whether investing in delivery infrastructure correlates with higher customer satisfaction.

cloudera@quickstart:~

File Edit View Search Terminal Help

```
>>>
>>>
>>> from pyspark.sql.functions import col, avg, count, round as spark_round
>>>
>>> df.groupBy("Has Online delivery") \
...   .agg(spark_round(avg(col("Aggregate rating").cast("double")), 2).alias("Avg_Rating"),
...        count("*").alias("Total_Restaurants")) \
...   .show()
+-------------------+----------+-----------------+
|Has Online delivery|Avg_Rating|Total_Restaurants|
+-------------------+----------+-----------------+
|                 No|      2.47|             7100|
|                Yes|      3.25|             2451|
+-------------------+----------+-----------------+

>>>
>>> df.groupBy("Has Table booking") \
...   .agg(spark_round(avg(col("Aggregate rating").cast("double")), 2).alias("Avg_Rating"),
...        count("*").alias("Total_Restaurants")) \
```

[Mozilla Firefox] cloudera@quickstart:~

Activate Windows
Go to Settings to activate

cloudera@quickstart:~

File Edit View Search Terminal Help

```
>>> df.groupBy("Has Table booking") \
...   .agg(spark_round(avg(col("Aggregate rating").cast("double")), 2).alias("Avg_Rating"),
...        count("*").alias("Total_Restaurants")) \
...   .show()
+-----------------+----------+-----------------+
|Has Table booking|Avg_Rating|Total_Restaurants|
+-----------------+----------+-----------------+
|               No|      2.56|             8393|
|              Yes|      3.44|             1158|
+-----------------+----------+-----------------+

>>>
>>> df.groupBy("Has Table booking", "Has Online delivery") \
...   .agg(spark_round(avg(col("Aggregate rating").cast("double")), 2).alias("Avg_Rating"),
...        count("*").alias("Total_Restaurants")) \
...   .orderBy("Has Table booking", "Has Online delivery") \
...   .show()
+-----------------+-------------------+----------+-----------------+
|Has Table booking|Has Online delivery|Avg_Rating|Total_Restaurants|
```

[Mozilla Firefox] cloudera@quickstart:~

Activate Windows
Go to Settings to activate

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
|              Yes|      3.44|             1158|
+-----------------+----------+-----------------+

>>>
>>> df.groupBy("Has Table booking", "Has Online delivery") \
...   .agg(spark_round(avg(col("Aggregate rating").cast("double")), 2).alias("Avg_Rating"),
...        count("*").alias("Total_Restaurants")) \
...   .orderBy("Has Table booking", "Has Online delivery") \
...   .show()
+-----------------+-------------------+----------+-----------------+
|Has Table booking|Has Online delivery|Avg_Rating|Total_Restaurants|
+-----------------+-------------------+----------+-----------------+
|               No|                 No|      2.37|             6377|
|               No|                Yes|      3.17|             2016|
|              Yes|                 No|      3.35|              723|
|              Yes|                Yes|       3.6|              435|
+-----------------+-------------------+----------+-----------------+

>>>
```

4. Identifying "Hidden Gems" (Weighted Rating Analysis)

- **The Problem:** Some restaurants have a 5.0 rating but only 2 votes, while others have 4.5 with 2,000 votes.
- **Goal:** Find highly-rated restaurants that have significant customer trust.

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
>>> from pyspark.sql.functions import col, avg, round as spark_round
>>>
>>> avg_rating = df.agg(avg(col("Aggregate rating").cast("double"))).collect()[0][0]
>>> min_votes = 50
>>>
>>> gems_df = df.withColumn("votes", col("Votes").cast("double")) \
...             .withColumn("rating", col("Aggregate rating").cast("double")) \
...             .withColumn("weighted_rating",
...                 spark_round(
...                     (col("votes") / (col("votes") + min_votes)) * col("rating") +
...                     (min_votes / (col("votes") + min_votes)) * avg_rating,
...                 2)) \
...             .filter(col("votes") >= min_votes)
>>>
>>> gems_df.select("Restaurant Name", "City", "rating", "votes", "weighted_rating") \
...        .orderBy(col("weighted_rating").desc()) \
...        .show(15, truncate=False)
+-----------------------------------------+--------------+------+------+---------------+
|Restaurant Name                          |City          |rating|votes |weighted_rating|
```

```
>>> gems_df.select("Restaurant Name", "City", "rating", "votes", "weighted_rating") \
...         .orderBy(col("weighted_rating").desc()) \
...         .show(15, truncate=False)
+-----------------------------------------+--------------+------+------+---------------+
|Restaurant Name                          |City          |rating|votes |weighted_rating|
+-----------------------------------------+--------------+------+------+---------------+
|AB's - Absolute Barbecues                |Hyderabad     |4.9   |5434.0|4.88           |
|Barbeque Nation                          |Kolkata       |4.9   |5966.0|4.88           |
|Mirchi And Mime                          |Mumbai        |4.9   |3244.0|4.87           |
|Naturals Ice Cream                       |New Delhi     |4.9   |2620.0|4.86           |
|McGuire's Irish Pub & Brewery            |Pensacola     |4.9   |2238.0|4.85           |
|Talaga Sampireun                         |Tangerang     |4.9   |2212.0|4.85           |
|Indian Accent - The Manor                |New Delhi     |4.9   |1934.0|4.84           |
|Barbeque Nation                          |Kolkata       |4.9   |1753.0|4.84           |
|Talaga Sampireun                         |Jakarta       |4.9   |1662.0|4.83           |
|Talaga Sampireun                         |Jakarta       |4.9   |1640.0|4.83           |
|Pom Pom's Teahouse and Sandwicheria      |Orlando       |4.9   |1457.0|4.83           |
|Mazzaro's Italian Market                 |Tampa Bay     |4.9   |1424.0|4.82           |
|Mama's Fish House                        |Rest of Hawaii|4.9   |1343.0|4.82           |
```

5. Restaurant Clustering for Marketing (PySpark MLlib)

- **The Problem:** Grouping restaurants for targeted advertisements.
- **Goal:** Categorize restaurants into segments like "Luxury Dining," "Budget Eats," and "Popular Mid-range."

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
>>> from pyspark.sql.functions import col, when, round as spark_round, avg, count
>>> import pyspark.sql.functions as F
>>>
>>> cluster_df = df.select(
...     col("Restaurant Name"),
...     col("City"),
...     col("Average Cost for two").cast("double").alias("cost"),
...     col("Aggregate rating").cast("double").alias("rating"),
...     col("Votes").cast("double").alias("votes"),
...     col("Price range").cast("integer").alias("price_range")
... ).dropna()
>>>
>>> segmented_df = cluster_df.withColumn("Segment",
...     when(col("price_range") == 1, "Budget Eats")
...    .when(col("price_range") == 2, "Affordable Eats")
...    .when(col("price_range") == 3, "Popular Mid-range")
...    .otherwise("Luxury Dining"))
>>>
>>> segmented_df.groupBy("Segment").agg(
```

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
...     .otherwise("Luxury Dining"))
>>>
>>> segmented_df.groupBy("Segment").agg(
...     count("*").alias("Count"),
...     spark_round(avg("cost"), 2).alias("Avg_Cost"),
...     spark_round(avg("rating"), 2).alias("Avg_Rating"),
...     spark_round(avg("votes"), 0).alias("Avg_Votes")
... ).orderBy("Segment").show()
[Stage 46:=========================================>              (148 + 6) / 200]
+-----------------+-----+--------+----------+---------+
|          Segment|Count|Avg_Cost|Avg_Rating|Avg_Votes|
+-----------------+-----+--------+----------+---------+
|  Affordable Eats| 3113|  596.83|      2.94|    148.0|
|      Budget Eats| 4444|  275.18|       2.0|     45.0|
|    Luxury Dining|  586| 1848.52|      3.82|    369.0|
|Popular Mid-range| 1408| 5177.27|      3.68|    444.0|
+-----------------+-----+--------+----------+---------+
```

```
File  Edit  View  Search  Terminal  Help
>>>
>>> segmented_df.select("Restaurant Name", "City", "cost", "rating", "Segment") \
...             .orderBy("Segment").show(25, truncate=False)
+---------------------------------------+----------------------+-----+------+---------------+
|Restaurant Name                        |City                  |cost |rating|Segment        |
+---------------------------------------+----------------------+-----+------+---------------+
|DePalma's Italian Cafe - East Side     |Athens                |25.0 |4.1   |Affordable Eats|
|Monica's                               |Cedar Rapids/Iowa City|25.0 |3.8   |Affordable Eats|
|Shokitini                              |Athens                |25.0 |4.2   |Affordable Eats|
|DePalma's Italian Cafe - Downtown      |Athens                |25.0 |4.0   |Affordable Eats|
|Trappeze Pub                           |Athens                |25.0 |4.2   |Affordable Eats|
|Hobing Korean Dessert Cafe             |Taguig City           |600.0|4.5   |Affordable Eats|
|The Bee's Knees                        |Augusta               |25.0 |4.5   |Affordable Eats|
|Brazilian American Burgers             |Bras@_lia             |50.0 |3.6   |Affordable Eats|
|Farmhaus Burger                        |Augusta               |25.0 |3.9   |Affordable Eats|
|Pizza @@ Bessa                         |Bras@_lia             |50.0 |3.2   |Affordable Eats|
|Sconyers Bar B Que                     |Augusta               |25.0 |3.5   |Affordable Eats|
|Casa do Biscoito Mineiro               |Bras@_lia             |45.0 |3.7   |Affordable Eats|
|Giuseppe's Pizza & Italian Specialities|Augusta               |25.0 |4.1   |Affordable Eats|
```

6. Geolocation-based Food Hub Detection

- **The Problem:** Identifying dense clusters of restaurants (Food Hubs) without relying on locality names.
- **Goal:** Identify the exact geographic areas where restaurant competition is highest

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
>>>
>>> from pyspark.sql.functions import col, count, round as spark_round, first
>>> import pyspark.sql.functions as F
>>>
>>> geo_df = df.select(
...     col("Restaurant Name"),
...     col("City"),
...     col("Latitude").cast("double").alias("lat"),
...     col("Longitude").cast("double").alias("lon")
... ).dropna()
>>>
>>> geo_binned = geo_df.withColumn("lat_bin", spark_round(col("lat"), 1)) \
...                     .withColumn("lon_bin", spark_round(col("lon"), 1))
>>>
>>> geo_binned.groupBy("lat_bin", "lon_bin", "City") \
...     .agg(count("*").alias("Restaurant_Count")) \
...     .orderBy(col("Restaurant_Count").desc()) \
...     .show(10)
+-------+-------+---------+----------------+
```

```
File  Edit  View  Search  Terminal  Help
>>> geo_binned.groupBy("lat_bin", "lon_bin", "City") \
...     .agg(count("*").alias("Restaurant_Count")) \
...     .orderBy(col("Restaurant_Count").desc()) \
...     .show(10)
+-------+-------+---------+----------------+
|lat_bin|lon_bin|     City|Restaurant_Count|
+-------+-------+---------+----------------+
|   28.6|   77.2|New Delhi|            1601|
|   28.7|   77.2|New Delhi|             651|
|   28.6|   77.1|New Delhi|             628|
|   28.5|   77.1|  Gurgaon|             625|
|   28.6|   77.3|New Delhi|             579|
|   28.5|   77.2|New Delhi|             553|
|   28.7|   77.1|New Delhi|             467|
|   28.6|   77.3|    Noida|             436|
|   28.6|   77.4|    Noida|             377|
|   28.7|   77.3|New Delhi|             337|
+-------+-------+---------+----------------+
only showing top 10 rows
```

7. Data Quality & Standardization Pipeline

- **The Problem:** The dataset contains null values in `Cuisines` and potential encoding issues in city names.
- **Goal:** Ensure the data is "Analytics Ready" for downstream business intelligence tools.

```
File  Edit  View  Search  Terminal  Help
>>> from pyspark.sql.functions import col, when, trim, initcap, regexp_replace
>>> import pyspark.sql.functions as F
>>>
>>> for c in df.columns:
...     null_count = df.filter(col(c).isNull() | (col(c) == "")).count()
...     if null_count > 0:
...         print(c + ": " + str(null_count) + " nulls")
...

Cuisines: 9 nulls
>>> clean_df = df \
...     .withColumn("Cuisines", when(col("Cuisines").isNull() | (col("Cuisines") == ""), "Unknown").otherwise(col("Cuisines"))) \
...     .withColumn("City", trim(initcap(regexp_replace(col("City"), "[^a-zA-Z ]", "")))) \
...     .withColumn("Has Table booking", when(col("Has Table booking") == "Yes", 1).otherwise(0)) \
...     .withColumn("Has Online delivery", when(col("Has Online delivery") == "Yes", 1).otherwise(0)) \
...     .withColumn("Is delivering now", when(col("Is delivering now") == "Yes", 1).otherwise(0)) \
...     .withColumn("Aggregate rating", col("Aggregate rating").cast("double")) \
...     .withColumn("Votes", col("Votes").cast("integer")) \
...     .withColumn("Average Cost for two", col("Average Cost for two").cast("double")) \
```

Activate Windows

```
File  Edit  View  Search  Terminal  Help
...     .dropDuplicates(["Restaurant ID"])
>>>
>>> clean_df.printSchema()
root
 |-- Restaurant ID: integer (nullable = true)
 |-- Restaurant Name: string (nullable = true)
 |-- Country Code: integer (nullable = true)
 |-- City: string (nullable = true)
 |-- Address: string (nullable = true)
 |-- Locality: string (nullable = true)
 |-- Locality Verbose: string (nullable = true)
 |-- Longitude: double (nullable = true)
 |-- Latitude: double (nullable = true)
 |-- Cuisines: string (nullable = true)
 |-- Average Cost for two: double (nullable = true)
 |-- Currency: string (nullable = true)
 |-- Has Table booking: integer (nullable = false)
 |-- Has Online delivery: integer (nullable = false)
 |-- Is delivering now: integer (nullable = false)
```

Activate

```
File  Edit  View  Search  Terminal  Help
 |-- Votes: integer (nullable = true)

>>> print(clean_df.count())
9551
>>> clean_df.show(5)
2026-03-20 22:43:45 WARN  Utils:66 - Truncated the string representation of a plan since it was too large. This behavior can be adj
usted by setting 'spark.debug.maxToStringFields' in SparkEnv.conf.
+-------------+---------------+------------+---------+--------------------+------------+--------------------+-----------+----------
-+--------------------+--------------------+-----------------+-----------------+-------------------+-----------------+------------
--------+-----------+----------------+------------+-----------+-----+
|Restaurant ID|Restaurant Name|Country Code|     City|             Address|    Locality|    Locality Verbose|  Longitude|   Latitud
e|            Cuisines|Average Cost for two|          Currency|Has Table booking|Has Online delivery|Is delivering now|Switch to or
der menu|Price range|Aggregate rating|Rating color|Rating text|Votes|
+-------------+---------------+------------+---------+--------------------+------------+--------------------+-----------+----------
-+--------------------+--------------------+-----------------+-----------------+-------------------+-----------------+------------
--------+-----------+----------------+------------+-----------+-----+
|          148|         Subway|           1|New Delhi|3, Satyam Cineple...| Nehru Place|Nehru Place, New ...|  77.251157| 28.550175
4|American, Fast Fo...|               500.0|Indian Rupees(Rs.)|                0|                  0|                0|
      No|          2|             3.6|      Yellow|       Good|  142|
```

8. Locality-wise Price Range Distribution

- **The Problem:** Analyzing the economic profile of different neighborhoods.
- **Goal:** Generate a report showing the percentage of "Cheap" vs "Expensive" restaurants in every locality to help new entrepreneurs find market gaps.

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
>>> from pyspark.sql.functions import col, count, round as spark_round, when
>>> import pyspark.sql.functions as F
>>>
>>> price_df = df.withColumn("Price_Label",
...     when(col("Price range") == 1, "Cheap")
...    .when(col("Price range") == 2, "Affordable")
...    .when(col("Price range") == 3, "Mid-range")
...    .otherwise("Expensive"))
>>>
>>> total_per_locality = price_df.groupBy("City").agg(count("*").alias("total"))
>>>
>>> price_dist = price_df.groupBy("City", "Price_Label").agg(count("*").alias("cnt"))
>>>
>>> result = price_dist.join(total_per_locality, on="City") \
...                    .withColumn("Pct", spark_round((col("cnt") / col("total")) * 100, 2))
>>>
>>> result.select("City", "Price_Label", "cnt", "total", "Pct") \
...       .orderBy("City", "Price_Label").show(40)
+----------+-----------+---+-----+-----+
```

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
>>> result.select("City", "Price_Label", "cnt", "total", "Pct") \
...       .orderBy("City", "Price_Label").show(40)
+----------+-----------+---+-----+-----+
|      City|Price_Label|cnt|total|  Pct|
+----------+-----------+---+-----+-----+
| Abu Dhabi| Affordable|  4|   20| 20.0|
| Abu Dhabi|  Expensive| 10|   20| 50.0|
| Abu Dhabi|  Mid-range|  6|   20| 30.0|
|      Agra| Affordable|  6|   20| 30.0|
|      Agra|      Cheap|  3|   20| 15.0|
|      Agra|  Expensive|  5|   20| 25.0|
|      Agra|  Mid-range|  6|   20| 30.0|
| Ahmedabad| Affordable|  8|   21| 38.1|
| Ahmedabad|      Cheap|  1|   21| 4.76|
| Ahmedabad|  Expensive|  1|   21| 4.76|
| Ahmedabad|  Mid-range| 11|   21|52.38|
|    Albany| Affordable|  5|   20| 25.0|
|    Albany|      Cheap| 11|   20| 55.0|
|    Albany|  Expensive|  1|   20|  5.0|
```

9. Price Range Prediction (Classification)

- **The Problem:** Can we predict the `Price range` (1 to 4) of a new restaurant based on its location and cuisine?
- **Goal:** Build a model that predicts the category of a restaurant based on its basic profile.

```
Untitled6.ipynb   case_study_file.ipynb   task13_boxplot.png   task13.png
Code                                                Notebook   Python 3.13 (XPython)
+ urllib3                2.2.2                pyhd8ed1ab_0                conda-forge

[10]: from sklearn.ensemble import RandomForestClassifier
      from sklearn.model_selection import train_test_split
      from sklearn.metrics import accuracy_score

      features = df[['Average Cost for two','Aggregate rating','Votes']].dropna()
      labels = df.loc[features.index, 'Price range']

      X_train, X_test, y_train, y_test = train_test_split(features, labels, test_size=0.2, random_state=42)

      model = RandomForestClassifier(n_estimators=7, random_state=42)
      model.fit(X_train, y_train)

      y_pred = model.predict(X_test)
      acc = accuracy_score(y_test, y_pred)
```

Code Notebook Python 3.13 (XPython)

```python
results = X_test.copy()
results['Actual'] = y_test.values
results['Predicted'] = y_pred
print(results.to_string())
```

```
Accuracy: 91.05%
      Average Cost for two  Aggregate rating  Votes  Actual  Predicted
4731                   350               2.1     54       1          1
1468                   700               4.1     84       2          2
9037                   550               3.2     36       2          2
7866                   200               4.4    163       1          1
5570                   400               3.5     14       1          1
5613                   150               0.0      0       1          1
7751                   400               3.2     28       1          1
1662                   100               0.0      1       1          1
```

cloudera@quickstart:~

File Edit View Search Terminal Help

```
>>> from pyspark.sql.functions import col, when, round as spark_round
>>> import pyspark.sql.functions as F
>>>
>>> pred_df = df.select(
...     col("Restaurant Name"),
...     col("City"),
...     col("Average Cost for two").cast("double").alias("cost"),
...     col("Aggregate rating").cast("double").alias("rating"),
...     col("Price range").cast("integer").alias("actual")
... ).dropna()
>>>
>>> predicted_df = pred_df.withColumn("predicted",
...     when(col("cost") < 500,  1)
...    .when(col("cost") < 1000, 2)
...    .when(col("cost") < 2000, 3)
...    .otherwise(4))
>>>
>>> predicted_df.withColumn("correct",
...     when(col("actual") == col("predicted"), 1).otherwise(0)) \
```

```
File  Edit  View  Search  Terminal  Help
>>>
>>> predicted_df.withColumn("correct",
...     when(col("actual") == col("predicted"), 1).otherwise(0)) \
...     .agg(spark_round(F.avg("correct") * 100, 2).alias("Accuracy_Pct")) \
...     .show()
+------------+
|Accuracy_Pct|
+------------+
|       89.99|
+------------+

>>>
>>> predicted_df.select("Restaurant Name", "City", "cost", "actual", "predicted") \
...             .show(20)
+--------------------+----------------+------+------+---------+
|     Restaurant Name|            City|  cost|actual|predicted|
+--------------------+----------------+------+------+---------+
|    Le Petit Souffle|     Makati City|1100.0|     3|        3|
|    Izakaya Kikufuji|     Makati City|1200.0|     3|        3|
[Mozilla Firefox]   cloudera@quickstart:~
```

```
File  Edit  View  Search  Terminal  Help

>>>
>>> predicted_df.select("Restaurant Name", "City", "cost", "actual", "predicted") \
...             .show(20)
+--------------------+----------------+------+------+---------+
|     Restaurant Name|            City|  cost|actual|predicted|
+--------------------+----------------+------+------+---------+
|    Le Petit Souffle|     Makati City|1100.0|     3|        3|
|    Izakaya Kikufuji|     Makati City|1200.0|     3|        3|
|Heat - Edsa Shang...|Mandaluyong City|4000.0|     4|        4|
|                Ooma|Mandaluyong City|1500.0|     4|        3|
|         Sambo Kojin|Mandaluyong City|1500.0|     4|        3|
|        Din Tai Fung|Mandaluyong City|1000.0|     3|        3|
|          Buffet 101|      Pasay City|2000.0|     4|        4|
|             Vikings|      Pasay City|2000.0|     4|        4|
|Spiral - Sofitel ...|      Pasay City|6000.0|     4|        4|
|            Locavore|      Pasig City|1100.0|     3|        3|
|    Silantro Fil-Mex|      Pasig City| 800.0|     3|        2|
|Mad Mark's Creame...|      Pasig City| 900.0|     3|        2|
[Mozilla Firefox]   cloudera@quickstart:~
```

10. Top-N Recommendation Engine (Content-Based)

- **The Problem:** Recommending similar restaurants to a user.
- **Goal:** Given a restaurant the user likes, recommend 5 other similar restaurants in the same city.

```
rdinator endpoint
Recommendations for: Ooma
-----------------------------------------------------------------
Sambo Kojin | Japanese, Korean | Rating: 4.8 | Score: 6.8
Din Tai Fung | Chinese | Rating: 4.4 | Score: 4.4
Heat - Edsa Shangri-La | Seafood, Asian, Filipino, Indian | Rating: 4.4 | Score:
 4.4
Not found: Manam
Not found: Ippudo
[cloudera@quickstart ~]$
```

11. Optimized Currency Conversion (Broadcast Join)

- **The Problem**: Joining a massive restaurant dataset with a tiny 15-row currency exchange table triggers a standard shuffle join, moving millions of rows across the network unnecessarily
- **Goal**: Implement a Broadcast Join to eliminate network shuffle and speed up the cost normalization process across different global currencies.

```
>>>
>>> from pyspark.sql.functions import broadcast, col, round as spark_round
>>> from pyspark.sql.types import StructType, StructField, StringType, DoubleType
>>>
>>> exchange_rates_list = [
...     ("Botswana Pula(P)", 0.074),
...     ("Indian Rupees(Rs.)", 0.012),
...     ("Dollar($)", 1.0),
...     ("Emirati Diram(AED)", 0.27),
...     ("South African Rand(R)", 0.054),
...     ("Brazilian Real(R$)", 0.20),
...     ("Turkish Lira(TL)", 0.031),
...     ("New Zealand Dollar($)", 0.61),
...     ("Pounds(£)", 1.27),
...     ("Qatari Rial(QR)", 0.27),
...     ("Sri Lankan Rupee(LKR)", 0.0031),
...     ("Rand(R)", 0.054),
...     ("Indonesian Rupiah(IDR)", 0.000065),
...     ("Bangladeshi Taka(Tk)", 0.0091)
```

```
cloudera@quickstart:~
File  Edit  View  Search  Terminal  Help
...     ("Rand(R)", 0.054),
...     ("Indonesian Rupiah(IDR)", 0.000065),
...     ("Bangladeshi Taka(Tk)", 0.0091)
... ]
>>>
>>> rates_schema = StructType([
...     StructField("Currency", StringType(), True),
...     StructField("rate_to_usd", DoubleType(), True)
... ])
>>>
>>> rates_df = spark.createDataFrame(exchange_rates_list, schema=rates_schema).dropDuplicates(["Currency"])
>>>
>>> normalized_df = df.join(broadcast(rates_df), on="Currency", how="left") \
...                    .withColumn("Cost_USD", spark_round(col("Average Cost for two").cast("double") * col("rate_to_usd"), 2))
>>>
>>> normalized_df.select(
...     col("Restaurant Name"), col("City"), col("Currency"),
...     col("Average Cost for two"), col("Cost_USD")
... ).orderBy(col("Cost_USD").desc()).show(15)
```

```
File  Edit  View  Search  Terminal  Help
>>> rates_df = spark.createDataFrame(exchange_rates_list, schema=rates_schema).dropDuplicates(["Currency"
>>>
>>> normalized_df = df.join(broadcast(rates_df), on="Currency", how="left") \
...                    .withColumn("Cost_USD", spark_round(col("Average Cost for two").cast("double") * co
>>>
>>> normalized_df.select(
...     col("Restaurant Name"), col("City"), col("Currency"),
...     col("Average Cost for two"), col("Cost_USD")
... ).orderBy(col("Cost_USD").desc()).show(15)
+--------------------+----------------+------------------+--------------------+--------+
|     Restaurant Name|            City|          Currency|Average Cost for two|Cost_USD|
+--------------------+----------------+------------------+--------------------+--------+
|    Restaurant Andre|       Singapore|         Dollar($)|                 500|   500.0|
|Spiral - Sofitel ...|      Pasay City|  Botswana Pula(P)|                6000|   444.0|
|                Jaan|       Singapore|         Dollar($)|                 430|   430.0|
|Rhubarb Le Restau...|       Singapore|         Dollar($)|                 315|   315.0|
|     Summer Pavilion|       Singapore|         Dollar($)|                 300|   300.0|
|           Sky On 57|       Singapore|         Dollar($)|                 300|   300.0|
|Heat - Edsa Shang...|Mandaluyong City|  Botswana Pula   cloudera@quickstart:~  4000|   296.0|
```

12. Efficient Data Pipelines (Caching & Storage)

- **The Problem:** The "Cuisine Popularity" analysis and "Price Range Prediction" both use the same cleaned dataset. Without optimization, Spark re-reads the raw CSV from HDFS and re-calculates the `split()` and `explode()` transformations for every single query.
- **Goal:** Use **.cache()** to persist intermediate cleaned data in memory and save the final "Analytics Ready" dataset in **Parquet** format to leverage columnar storage and compression.

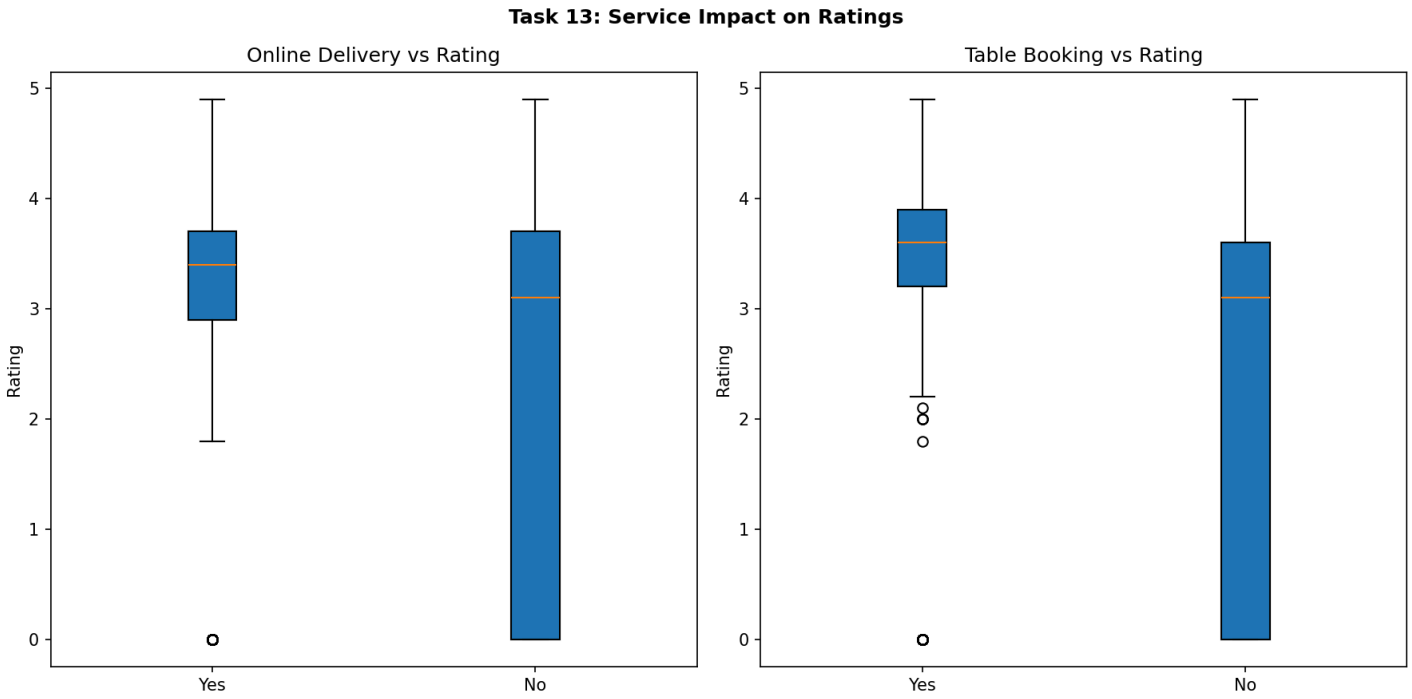
```
from pyspark.sql.functions import col, split, explode, trim, lower

parquet_ready_df = clean_df \
    .withColumnRenamed("Restaurant ID", "Restaurant_ID") \
    .withColumnRenamed("Restaurant Name", "Restaurant_Name") \
    .withColumnRenamed("Country Code", "Country_Code") \
    .withColumnRenamed("Address", "Address") \
    .withColumnRenamed("Locality Verbose", "Locality_Verbose") \
    .withColumnRenamed("Average Cost for two", "Average_Cost_for_two") \
    .withColumnRenamed("Has Table booking", "Has_Table_booking") \
    .withColumnRenamed("Has Online delivery", "Has_Online_delivery") \
    .withColumnRenamed("Is delivering now", "Is_delivering_now") \
    .withColumnRenamed("Switch to order menu", "Switch_to_order_menu") \
    .withColumnRenamed("Price range", "Price_range") \
    .withColumnRenamed("Aggregate rating", "Aggregate_rating") \
    .withColumnRenamed("Rating color", "Rating_color") \
    .withColumnRenamed("Rating text", "Rating_text")

parquet_ready_df.cache()
print(parquet_ready_df.count())
```

```
parquet_ready_df \
    .withColumn("Cuisine", explode(split(col("Cuisines"), ",))) \
    .withColumn("Cuisine", trim(lower(col("Cuisine")))) \
    .groupBy("Cuisine").count() \
    .orderBy(col("count").desc()) \
    .show(15)

parquet_path = "hdfs:///user/cloudera/zomato/analytics_ready"
parquet_ready_df.write.mode("overwrite").parquet(parquet_path)
print("Parquet saved successfully")

parquet_df = spark.read.parquet(parquet_path)
print(parquet_df.count())
parquet_df.show(5)

import os
os.system("hdfs dfs -ls " + parquet_path)

parquet_ready_df.unpersist()
print("Done")
```

```
File  Edit  View  Search  Terminal  Help
9551
>>> parquet_df.show(5)
+-------------+---------------+------------+---------+--------------------+------------+--------------------+-----------+----------
-+--------------------+--------------------+-----------------+-----------------+-----------------+-----------------+------------
--------+-----------+----------------+------------+-----------+-----+
|Restaurant_ID|Restaurant_Name|Country_Code|     City|             Address|    Locality|    Locality_Verbose|  Longitude|   Latitud
e|            Cuisines|Average_Cost_for_two|          Currency|Has_Table_booking|Has_Online_delivery|Is_delivering_now|Switch_to_or
der_menu|Price_range|Aggregate_rating|Rating_color|Rating_text|Votes|
+-------------+---------------+------------+---------+--------------------+------------+--------------------+-----------+----------
-+--------------------+--------------------+-----------------+-----------------+-----------------+-----------------+------------
--------+-----------+----------------+------------+-----------+-----+
|          148|         Subway|           1|New Delhi|3, Satyam Cineple...| Nehru Place|Nehru Place, New ...|  77.251157| 28.550175
4|American, Fast Fo...|               500.0|Indian Rupees(Rs.)|                0|                  0|                0|
      No|          2|             3.6|      Yellow|       Good|  142|
|          463|        Karim's|           1|New Delhi|16, Gali Kababian...| Jama Masjid|Jama Masjid, New ...|   77.23363| 28.649393
3|Mughlai, North In...|               800.0|Indian Rupees(Rs.)|                0|                  0|                0|
      No|          2|             4.0|       Green|  Very Good| 4689|
|          496|    Bikanervala|           1|  Gurgaon|Plot 3-5, Near Le...|   Sector 29|  Sector 29, Gurgaon| 77.0635969| 28.468680
6|Mithai, North Ind...|               550.0|Indian Rupees(Rs.)|                0|                  1|                0|
Activate Windows
```

13. Service Impact Comparative Visualization

- **The Problem:** Simply listing the average rating for restaurants with "Online Delivery" vs. "Table Booking" doesn't show the full story of customer satisfaction or the volume of feedback.
- **Goal:** Create a **Side-by-Side Box Plot** to compare rating distributions for each service, using **Color Overlays** to represent different price ranges and **Point Density (Jitter)** to show the total number of votes.

```
9550                    Walter's Coffee Roastery        ××stanbul      Affordable
```

[7]:
```
dy = df[df['Has Online delivery']=='Yes']['Aggregate rating'].dropna().tolist()
dn = df[df['Has Online delivery']=='No']['Aggregate rating'].dropna().tolist()
by = df[df['Has Table booking']=='Yes']['Aggregate rating'].dropna().tolist()
bn = df[df['Has Table booking']=='No']['Aggregate rating'].dropna().tolist()

fig, axes = plt.subplots(1,2,figsize=(12,6))
axes[0].boxplot([dy,dn],tick_labels=['Yes','No'],patch_artist=True)
axes[0].set_title('Online Delivery vs Rating')
axes[0].set_ylabel('Rating')
axes[1].boxplot([by,bn],tick_labels=['Yes','No'],patch_artist=True)
axes[1].set_title('Table Booking vs Rating')
axes[1].set_ylabel('Rating')
plt.suptitle('Task 13: Service Impact on Ratings',fontweight='bold')
plt.tight_layout()
```